

CHAPTER 3 : METHODOLOGY

3.1 Introduction

In this chapter, the author outlines the methodological basis of the creation of the Integrated Pilgrim Journey Management Application, including the drawbacks to static navigation and coordination in current Umrah applications (Alamoudi and Abd, 2025). Consistent with the project aims, including a client-server architecture, simulation of the pilgrim journeys based on geofencing, and effective data management, the Design Science Research (DSR) approach is taken. DSR is chosen due to its focus on generating novel artifacts to address the practical challenges by means of design and testing cycles, which makes it a viable choice in the stage of the proposal (FYP1) (Kumar, 2024).

The main artifact is a prototype of a mobile application based on the use of geofencing as the main triggering location mechanism that enables dynamic ritual guidance and progression with the help of adaptive scheduling (Abdoh, 2025). The method presents two possible operational modes: the manual mode, in which there is no restriction to accessing ritual areas, and the location-based mode, where geofencing limits continued movement to satisfy fixed orders, including requiring Miqat and niyyah detection before Tawaf is a gateway (Shambour & Gutub, 2022). The user experience will start with the entry into the application, where registration or login and location permission requests are made, and the experience will be adapted to user preferences to remain usable despite the refusal to grant any permission (Shevchenko and Reips, 2024). When the system is triggered by ritual selection or any other selected ritual, it automatically provides step-by-step instructions, such as du‘ā’, niyyah, and contextual instructions (Shukur et al., 2025). The chapter clarifies the

research plan, system architecture, tool rationales, UML modeling, database design, geofencing implementation plan and strategy to evaluate the plan of transitioning to FYP2.

3.2 Research Approach

The research methodology adopted in this project is the Design Science Research (DSR), which offers a rigorous approach to designing and assessing information technology artifacts to suit real-world issues, including cognitive overload and inefficient navigation during Umrah (vom Brocke et al., 2020). Specializing in geofencing as the main triggers, which determine the location-based method of triggering rituals and their development, DSR allows developing a practical prototype without losing the theoretical background.

3.2.1 Design Science Research Framework.

The DSR process is transformed to consist of three iterative steps to put geofencing as the central artifact on the agenda. The identification of the problem and motivation is the result of the literature analysis of Chapter 2, where it is mentioned that the applications of guiding rituals based on location and specific conditions do not exist (Shambour & Gutub, 2022). This creates the necessity of geofencing to determine virtual borders of holy places to facilitate an adaptive development process, and hybrid scheduling can be used as an auxiliary resource of itinerary changes.

The design and development stage, which is the focus of FYP1, includes the iterative modelling of the geofencing engine, the UML diagrams and ERD that depict the important aspects of the geofencing engine, like location triggers and mode-specific behaviour. The artifact is based on the approaches to geofencing, where two modes are used: manual to

provide easy entry and location-based to enforce the sequence with the assistance of mobile development principles (Shevchenko and Reips, 2024). The framework will make the design theoretically well-informed, and the geofencing will allow real-time reaction to the pilgrim geography, e.g., detection of Miqat prior to Tawaf.

An evaluation phase details a pre-implementation plan of FYP2, which will measure geofencing precision, trigger reliability, and usage of the mode in terms of such variables as timeliness of the notification and user satisfaction. This recursive strategy, as the proponents of DSR guidelines claim, facilitates optimization according to measurable results, which will add a new theoretically based solution to the problems of Umrah navigation (Kumar, 2024).

3.2.2 Requirements Elicitation Plan.

Requirements elicitation will make sure that the system is made to match pilgrim requirements that are based on systematic analysis of Umrah practices in official guides such as Panduan Lengkap Ibadah Umrah and Tabung Haji materials. Functional requirements put more emphasis on the features enabled by geofencing, with the following sequence of priorities:

1. FR-1 (Dual-Mode Operation): The system will provide both manual mode to access rituals without restrictions and location-based to perform geofenced progression (Shambour & Gutub, 2022).
2. FR-2 (Geofence Enforcement): When in location-based mode, Miqat and niyyah must be identified initially and Tawaf serves as a checkpoint to prohibit further rituals until it is completed.

3. FR-3 (Automatic Ritual Guidance): This is done if a trigger or selection is made, then step by step instructions must be offered, dua, niyyah and contextual information.
4. FR-4 (Itinerary Management): Support Adaptive scheduling is a secondary feature that helps to make real-time changes.

Non-functional requirements also focus on the privacy, as the location data collection and on-sensory processing will require specific permission to reduce exposure (Shevchenko and Reips, 2024). It incorporates performance and battery efficiency with the integration of fused location providers and usability with the provision of intuitive interfaces with a variety of users including the support of many languages. Mode flexibility supports a smooth transition to a different mode, and the functionality remains intact in case of permission refusal. FYP1 will have literature synthesis to validate requirements; FYP2 will use surveys and focus groups to conduct empirical validation (vom Brocke et al., 2020).

3.2.3 Ethical Considerations

With the sensitivity of the location data, the ethical design is taken into consideration at the beginning, and the privacy-by-design principles are followed. Location data will be anonymized to do analytics, and geofencing functionality should not be implemented without explicit and well-informed consent. The transmission of data is based on encryption protocols which are in the regulations such as GDPR and the Malaysian PDPA, which guarantees the trust of pilgrims and legal compliance. The system is user-respectful by giving full access to manual mode in case of location access denial or when users want to navigate freely with rituals, which is ethical and does not affect the functionality.

3.2.4 Software Development Methodology – Agile Approach

This project uses the Agile software development methodology to guide the design and prototyping of the Umrah Pilgrim Journey Management Application. Agile is chosen because the system involves continuous refinement of user requirements, interface design, and location-based notification logic. Since the application integrates mobile UI, database design, and geofencing functionality, an iterative development approach allows features to be developed and tested progressively rather than in a single rigid development cycle.

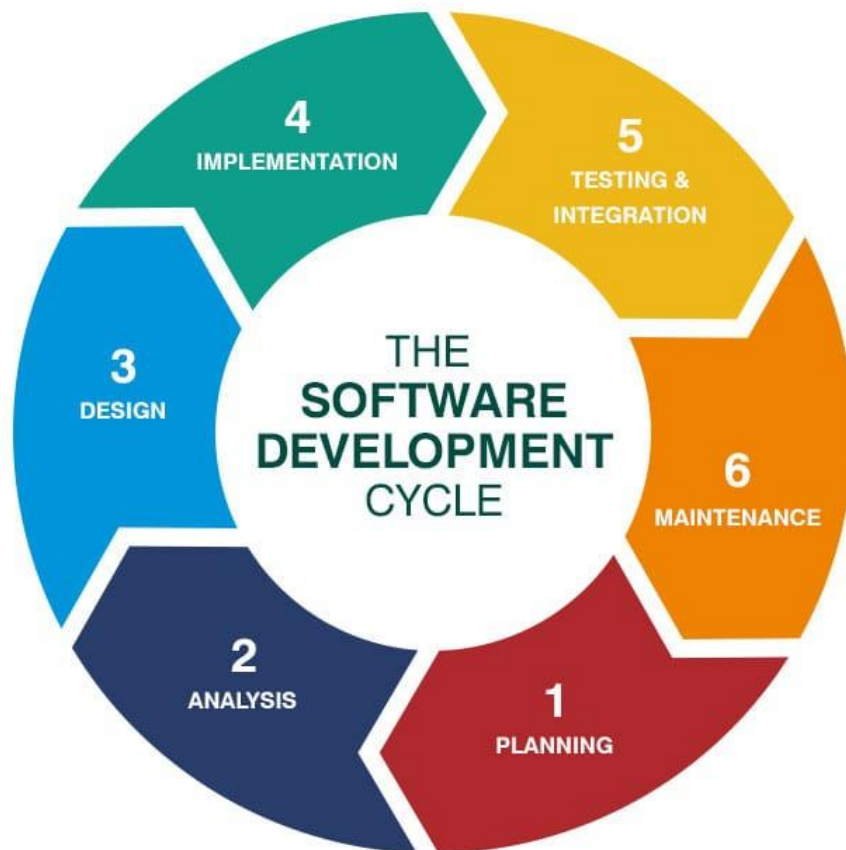


Figure 1: Software Development Cycle

The Agile process in this project is divided into multiple development iterations (sprints). Each sprint focuses on completing a specific functional component of the system, such as user registration, ritual-stage tracking, geofence detection, and notification triggering. Feedback from supervisors and test users is used to refine features before proceeding to the next iteration.

Currently, this project is in Phase 1 (Planning) and Phase 2 (Design) of the Agile cycle.

- **Phase 1 – Planning:**

In this phase, system requirements and functional objectives are identified. Key system features such as pilgrim profile management, ritual progress tracking, geofence-based location detection, and notification services are defined. Use case diagrams and system flowcharts are produced to clarify system interactions between users, mobile application interfaces, and the backend database.

- **Phase 2 – Design:**

In this phase, the system architecture and database structure are designed. The mobile application interface is planned using Flutter UI layouts, while the backend database is designed using MySQL to store pilgrim profiles, ritual stages, and geofence event logs. Sequence diagrams and ERD models are developed to represent data flow and system interactions. The geofencing logic is designed to trigger notification services when users enter specific ritual zones.

Future Agile phases, including Development, Testing, and Evaluation, will involve implementing the mobile application prototype, integrating geofence detection functions, and testing notification accuracy and database performance.

By adopting Agile methodology, the project ensures flexibility in refining system features and supports incremental development of the geofencing-based guidance system. This approach is suitable for mobile application projects where user experience and functional accuracy require continuous improvement.

3.3 Proposed System Architecture

The system employs a three-tier client server architecture to provide scalability, maintainability, and separation of concern as shown in Figure 1. The focus of this design is to put the geofencing mechanism at the core of location-based triggers, and the hybrid scheduling works as a support mechanism to adaptive changes (Vedantham, 2024).

Three-Tier Client-Server Model for Umrah Pilgrim Management

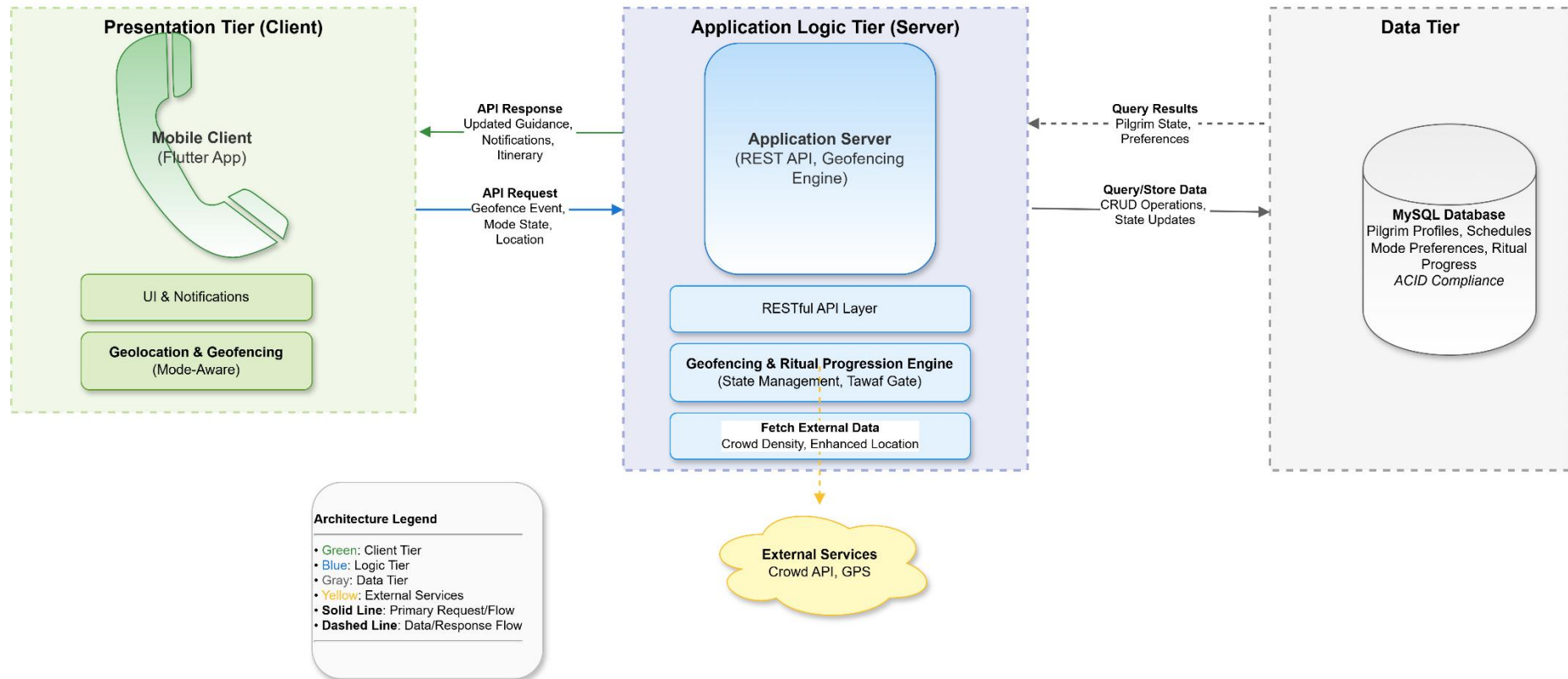


Figure 2 System Architecture

3.3.1 High-Level Client-server Architecture.

It has a three-layer architecture. The presentation layer (client-side) is developed based on Flutter, which gives the user-interference on Android and iOS gadgets to pilgrims. This level comes into effect when dealing with user interfaces and presenting custom itineraries and maps, local geofencing notifications through native APIs, including Android Geofencing API and iOS Core Location (Shevchenko and Reips, 2024). An integrated mode manager component is used here to allow a smooth transition between location-based and manual modes with the latter applying ritual sequencing via geofence detection. The location permission states are kept in the location, and the application modifies the functionality when permissions are denied, e.g., switching to manual mode, but does not change the usability (Kiasta et al., 2025).

The geofencing coordination engine is implemented on the application logic layer (server-side) and it consumes real time feeds such as crowd density and pilgrim locations through RESTful APIs. This level authenticates the progression of the ritual, e.g., the validation of Miqat and of niyyah preceding Tawaf in location mode, and activates scheduling support on-demand in optimizations (Hernandez-Lamas et al., 2021). Mode state management logic is used in order to guarantee enforcement on the location-based operations so that progression to other rituals does not occur unless Tawaf is completed.

The data layer which is handled by MySQL holds the structured data such as profile of pilgrims, schedule of rituals, geofence coordinates and mode preferences to maintain data integrity by compliance with ACID. Geofencing primacy is backed by architectural isolation to enable the logic tier dynamically combine both the static data and the contextual inputs and comply with the privacy standards (Vedantham, 2024).

3.3.2 Significant Functional Modules and Data Movement.

The functionality of the system is structured into prioritized modules, which have been represented by the interactions and data flows in Figure 2. The geofencing and location services component is central, that is, it determines the virtual fences surrounding sacred locations and creates entry/exit events to activate the progress of the ritual and behavioural-specific modes (Shambour and Gutub, 2022). It operates in location-based mode like the enforcement of sequencing, which is Tawaf as a checkpoint; in manual mode, it is non-responsive to unlimited access.

Ritual guidance module displays ritual content in the form of step-by-step instructions, du'a, niyyah, and context-specific details on geofence triggers or can be chosen manually (Kiasta et al., 2025). The authentication and authorization system guarantees that only pilgrims can access and log in to the system as well as role-based access. The adaptive scheduling module, which currently becomes secondary, modifies the plans according to geofencing information such as the crowd data. The notification and alert system spreads context-dependent information (messages), like the alert that a ritual is going on or user accidentally exits the boundaries while doing the ritual. The flow of data starts with the client-side use of location updates to the server, which is processed using database states and mode rules with the resulting response being updated guidance or notifications sent back to clients (Hernandez-Lomas et al., 2021).

Information Flow for Pilgrim Journey Management

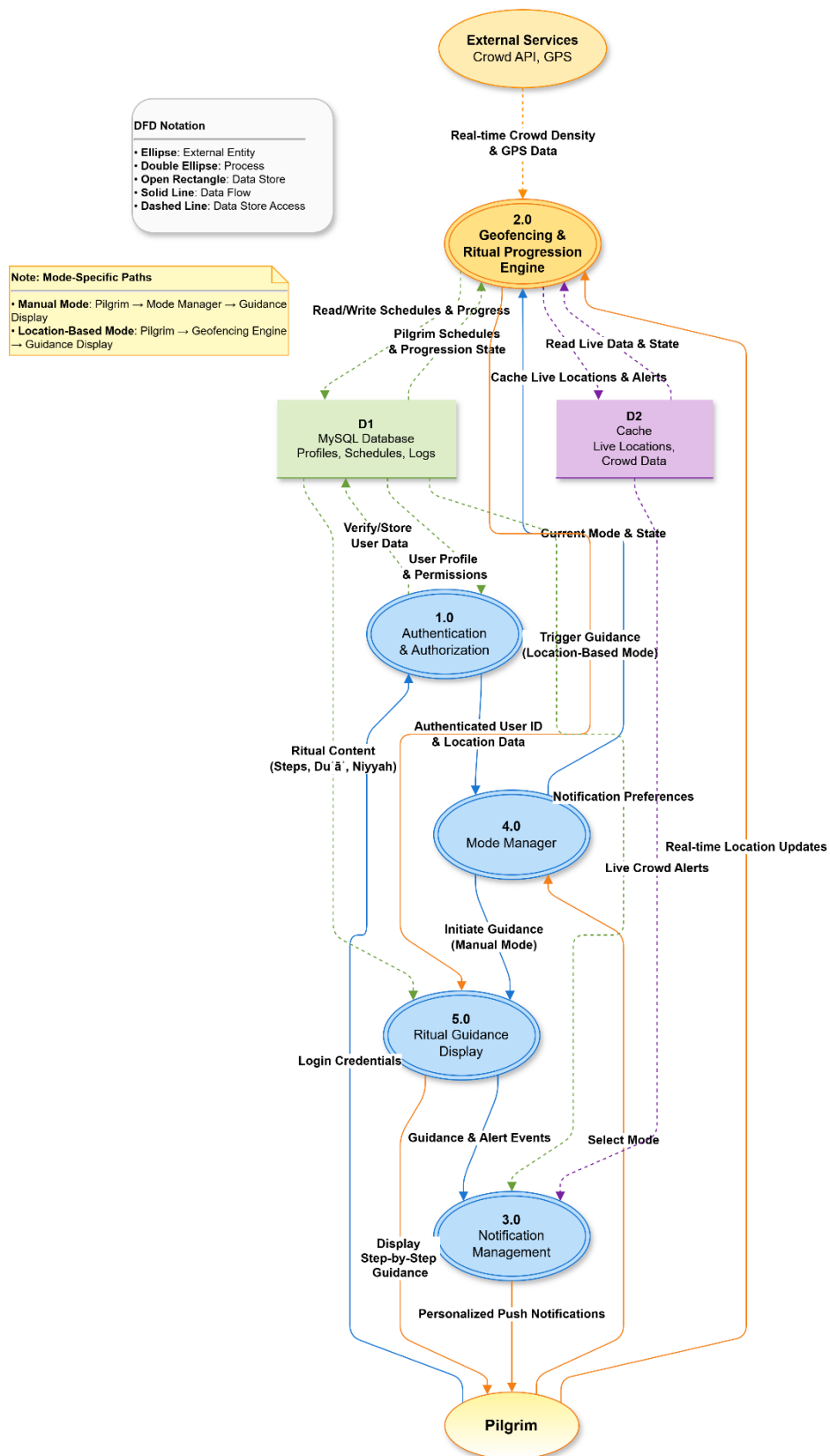


Figure 3 Data flow

3.4 Justification of Development Tools and Platform.

The choice of the development tools is based on the project goals of reaching high compatibility, direct management of data, and integration of geofencing, it is assumed to be focused on geofencing as the major system of location-based ritual guidance (Jošt & Taneski, 2025).

3.4.1 Front-end Cross-Platform Development with Flutter.

Flutter is selected as a front-end development tool because it has the benefits of cross-platform mobile application. Its single codebase paradigm will result in a shorter development time and costs and native-like performance on Android and iOS, which is required by pilgrims who have different devices (Diansyah and Syafrinal, 2025). The geolocation and mapping Flutter ecosystem contains mature owing to flutter_geofencing, which enables triggering of locations and mode-specific behaviour (Jošt & Taneski, 2025). Hot reload feature enables quick development of UI prototyping and modification in development with incorporation of stakeholder feedback in the design. Moreover, the stateful widget paradigm of Flutter allows making a switch between manual and location-based modes to operate efficiently, make it easy to accommodate geofence events and show automatic ritual guidance without a significant cost to battery performance (Shevchenko and Reips, 2024).

3.4.2 Back-end and Database MySQL with RESTful API.

The choice of MySQL as a relational database management system (RDBMS) to store the data is determined by the fact that the structure is appropriate to the core data of the

application, such as pilgrim profiles, schedules of rituals, and geofence coordinates (Vijayan, 2025). The ACID compliance of MySQL ensures data integrity of such critical processes as progression tracking and mode preferences and allows advanced queries to be created to support real-time decision-making (Alozie et al., 2025). MySQL is more consistent and supports querying, which are favoured over the alternatives based on the more basic operations of the system (Vijayan, 2025). The back-end is realised through a server-side software that delivers RESTful interfaces that guarantee a stateless, scaled, and cacheable client-server interface (Alozie et al., 2025).

Table 3: Rationale of the Tool Selection.

Component	Selected Tool	Primary Justification	Key Advantage for This Project
Front-end Framework	Flutter	Single codebase for iOS & Android	Faster development, consistent UI, geofencing plugins
Geolocation Services	flutter_geofencing, geolocator	Native API access, mature ecosystem	Accurate boundary detection, efficient battery usage, background monitoring
Database	MySQL	ACID compliance, relational structure	Data integrity for schedules, efficient complex queries
API Protocol	RESTful	Stateless, cacheable, standard	Scalability, ease of integration with Flutter
Mode Manager	Flutter State Management (Provider/Riverpod)	Real-time mode switching	Seamless transition, minimal latency

3.5 UML Diagrams and System Modelling.

Unified Modelling Language (UML) diagrams are used to express the system structure and behavior in a visual fashion and provide accurate communicated design requirements and design-quality requirements (Mishra, 2024).

3.5.1 Use Case Diagram

Figure 3 shows the use case diagram, which outlines major actors, who in this case are limited to the pilgrim and their interactions with the system functionalities.

Register/Login, Grant/Deny Location Permission, Select Manual Mode, Select Location-Based Mode, Access Ritual Guide Manually, Receive Location-Triggered Guidance, Complete Tawaf (checkpoint), View Personalized Itinerary, and Receive Notifications are the main applications. The system boundary is well defined, and includes relationships are clearly indicated with << includes >> relationships, mode dependent actions include location-based mode that includes geofence triggers to enforce ritual activities (Justinmind, 2024). The diagram explains the functional scope, which is directly related to elicited requirements as it shows mode flexibility and geofencing primacy.

System Functionality from User Perspective

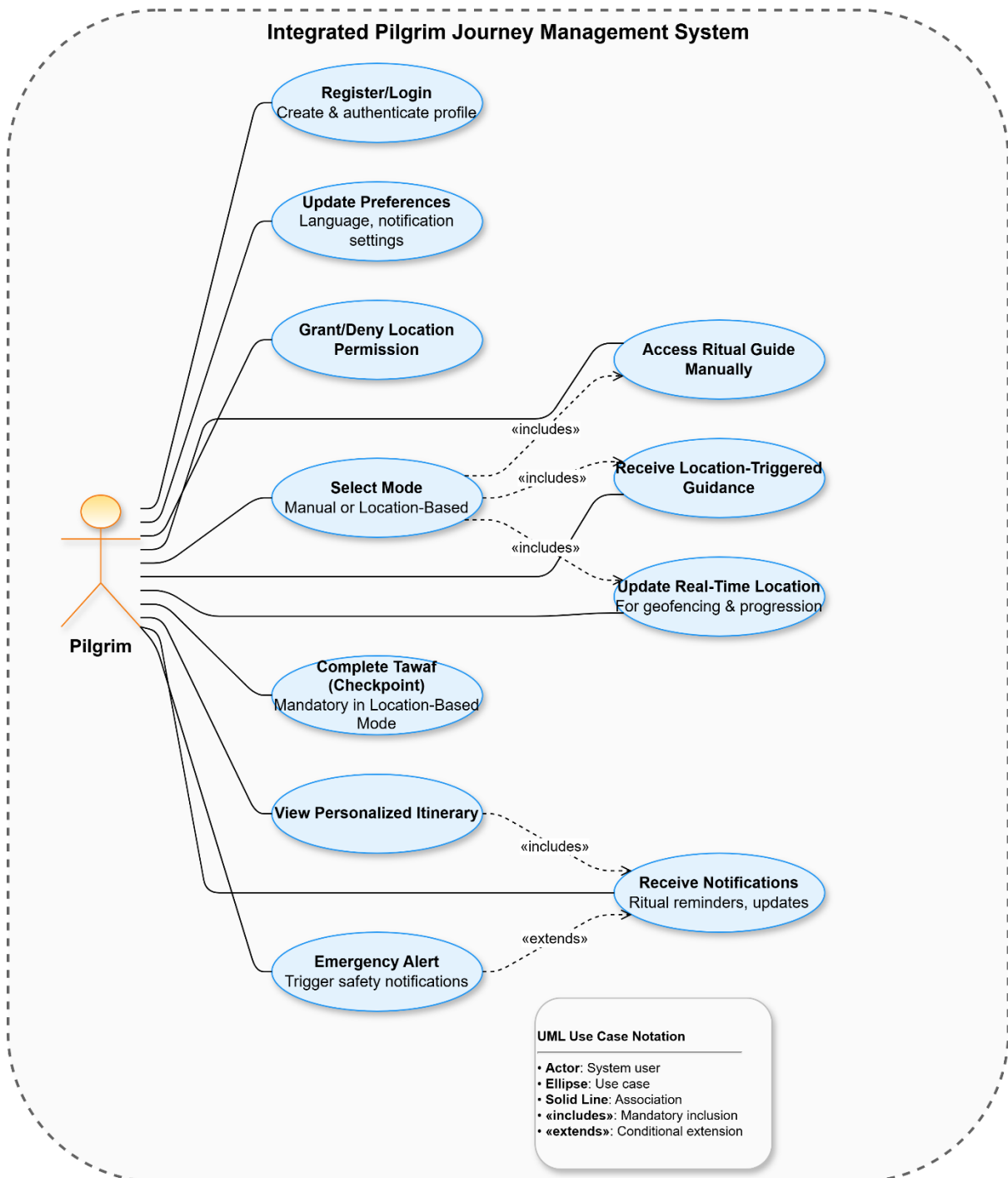


Figure 4 Use Case

3.5.2 Activity Diagram

The pilgrim journey is presented in Figure 4 as an activity diagram with lanes of pilgrim, system and external (GPS). The flow starts at the start of the application, moves to registration/login decisions, location permission requests (with continuity adaptability no

matter the choice) and mode selection. It splits into parallel flows whereby the manual mode lets one choose the ritual directly and do it self-reportedly; location-based mode triggers geofencing, imposes Miqat/niyyah detection, Tawaf as a mandatory checkpoint (decision: "Tawaf Completed?"), and transitions to the next rituals. The two modes have automatic guidance display, which converges at the end of ritual completion and fork/join notations of condition paths (GeeksforGeeks, 2025a). This visualization is dynamic, adaptive to the real-time circumstances, including the decisions of crowds density, and it highlights the relevance of geofencing in the imposed sequencing with manual flexibility to user autonomy (Shevchenko and Reips, 2024).

Figure 3.4: Activity Diagram - Pilgrim Journey with Dual Modes

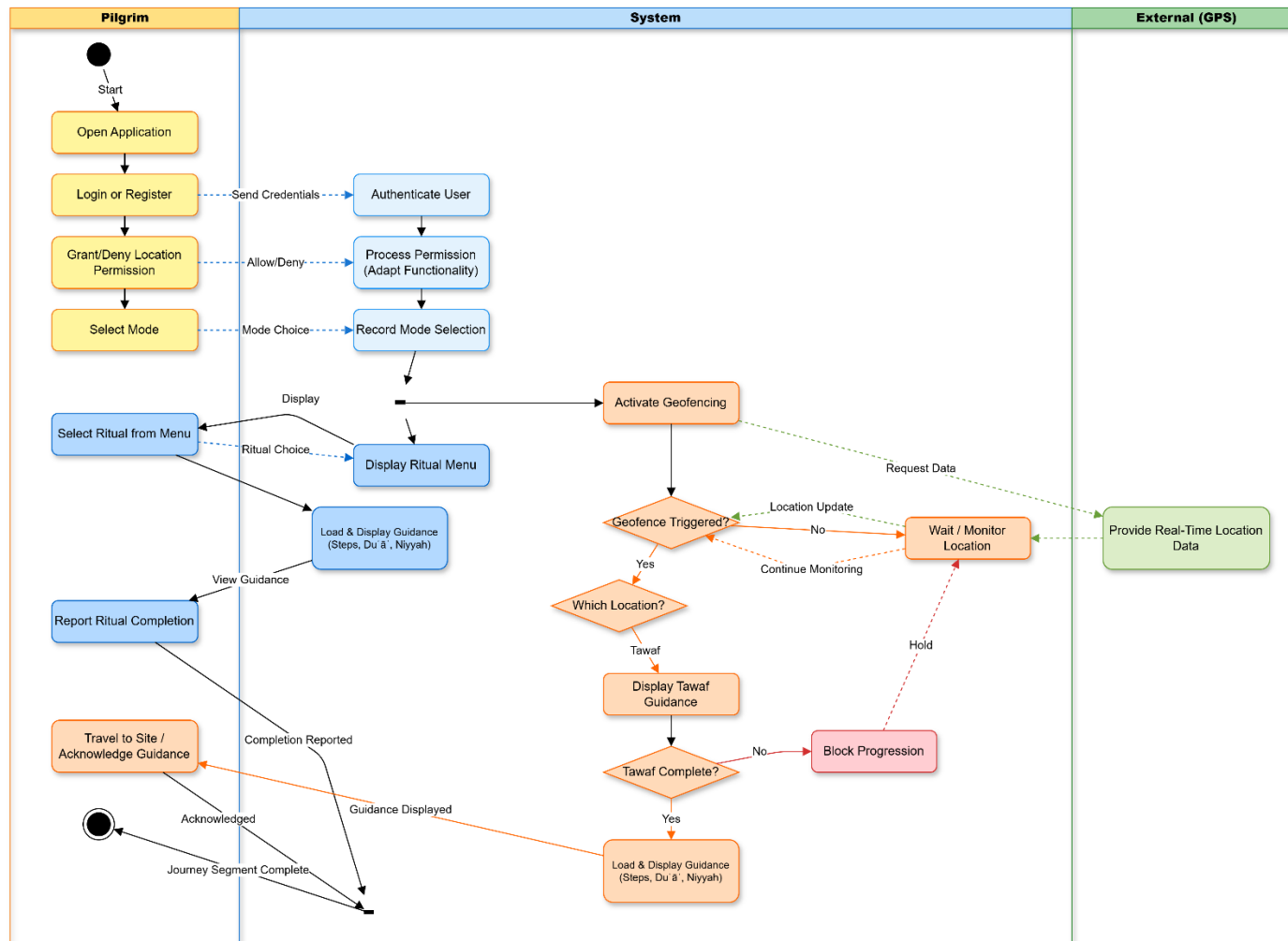


Figure 5 Activity diagram

3.5.3 Sequence Diagram

The sequence diagram (Fig. 4) shows the sequence diagram of a geofence triggered notification in the location-based mode, with lifelines of Pilgrim Device, Geofencing Service, Application Server and Notification Service. Geofence entry recognition triggers off the interaction and server-side processing that includes pilgrim schedule and preference querying, crowd density retrieval and mode checks. An alt fragment is used when the crowdDensity exceeds threshold (high density), routing is optimized, and a loop is used to check the active journeys after every five minutes. All the messages, including returning ones, are always marked, and personalized pushed messages are the final point (Visual Paradigm, 2023). Such a model provides rational timeliness of reminders with geofencing as the initiator. In case of manual mode, there is an additional description, which describes a more simplified process: a pilgrim directly selects a ritual without invoking geofence triggers, and the guidance content is obtained and shown by the server. The diagram complies with the standards of UML and rectifies the earlier design problems on accuracy (Mishra, 2024).

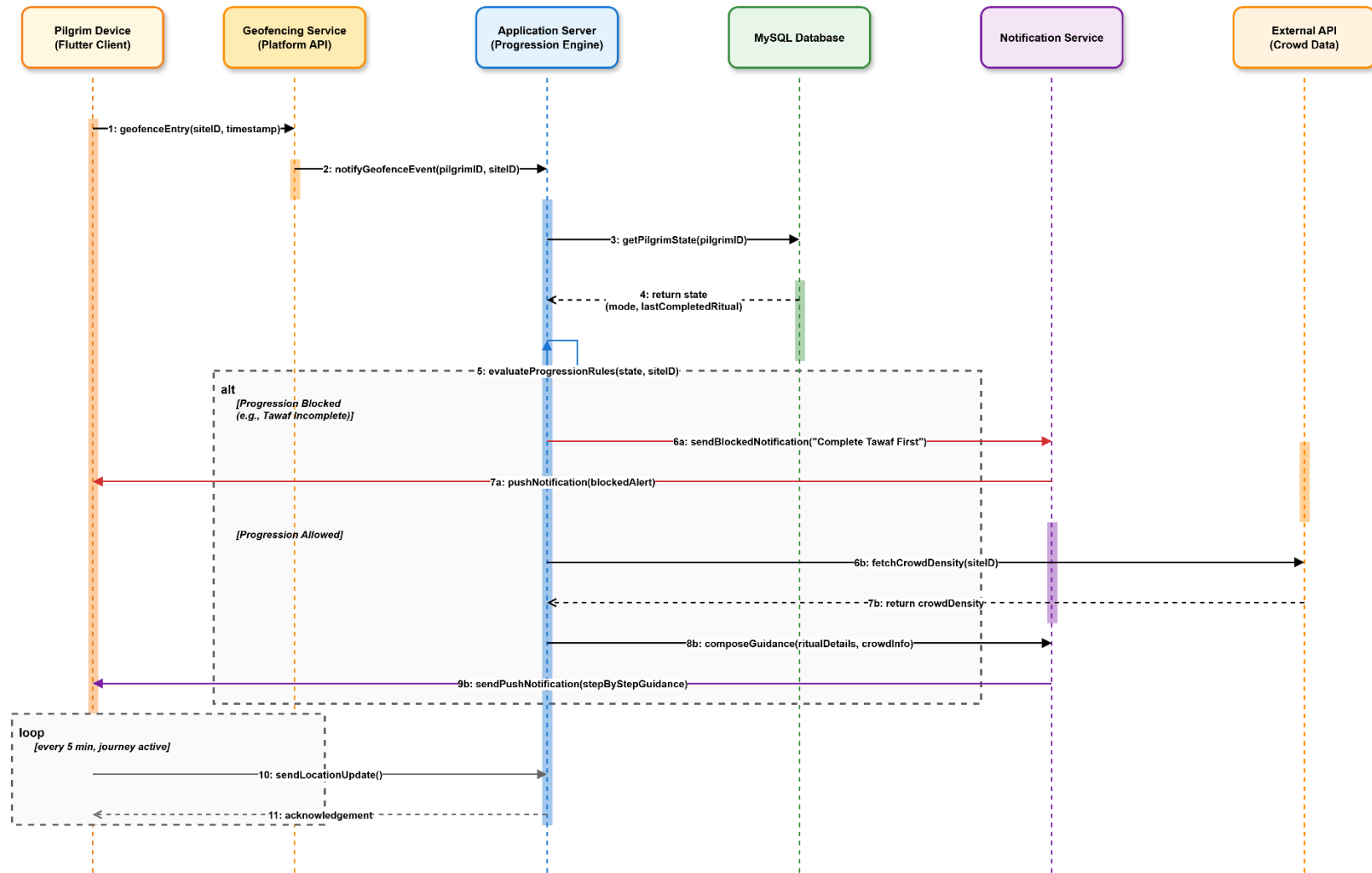


Figure 6 Sequence diagram

3.5.4 User Journey Flowchart

The User Journey and Ritual Progression Flowchart (figure 3.6) symbolizes the interaction of the pilgrim since entering the application and switching the algorithms that are focused on the backend and switching to the focus on the real-world journey (GeeksforGeeks, 2024). It is designed in form of a flowchart, beginning with User Opens App, which leads to registration/login decisions, followed by location permission requests (adaptively repeated in case of refusal). One of these decision nodes is a key decision node, defined as Which Mode, further divided into columns: manual-based, loading step-by-step guidance (including du‘ā’ and niyyah) on a choice, and the user reports completion, with a loop that allows repeating the Tawaf ritual; location-based, which enables geofencing and watches triggers (as in Miqat detection), and imposes a checkpoint of Tawaf as required (which requires the user to answer Tawaf Complete?) stopping progress on any incompleteness and automatically showing contextual instructions on sequential rituals such as Sa‘i. The two paths meet at the completion summary, and the colour coding, blue(manual), orange(location-based), green(shared processes), and yellow(decisions) are used, and a symbol legend is included to make the paths understandable (GeeksforGeeks, 2025b). This flowchart is a realization of dual-mode architecture and emphasizes the implementation of geofencing within location-based routes, maintaining the flexibility of manual means, which is an explicit response to the usability and progression needs (Shevchenko and Reips, 2024).

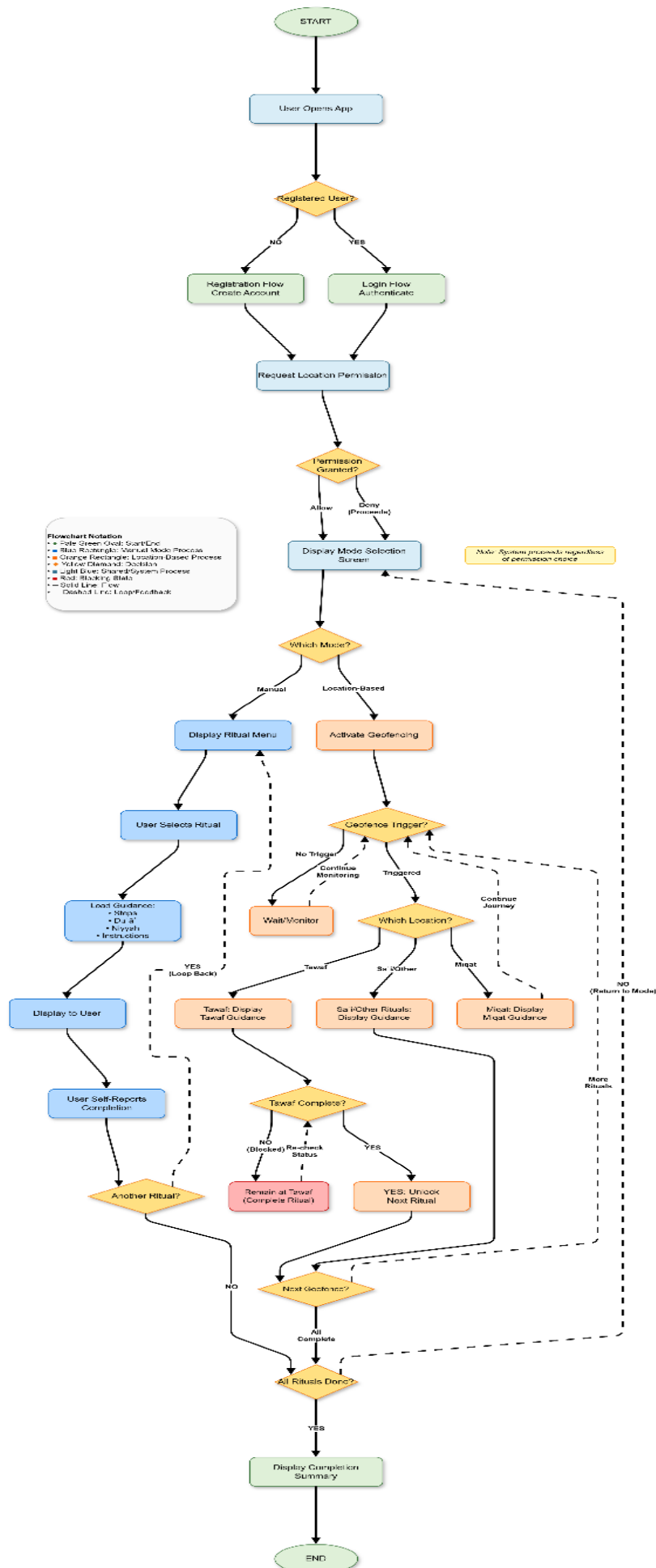


Figure 7: User journey and ritual progression flowchart

3.5.5 Geofencing Logic Flowchart

Additional geofencing logic flowchart, which elaborates on detection at the backend is given in Figure 6 to augment the user journey. It defines geofence registration (i.e., site coordinates and radii), entry/exit observation, trigger conditions (i.e., dwell time) and server-side notification streams. Mode state and progression flag evaluation in decision nodes only use scheduling support to perform optimizations such as crowd rerouting. This technical representation isolates the algorithmic functionality and interactions between the user, as geofencing is reliable in location-based mode (Vedantham, 2024).

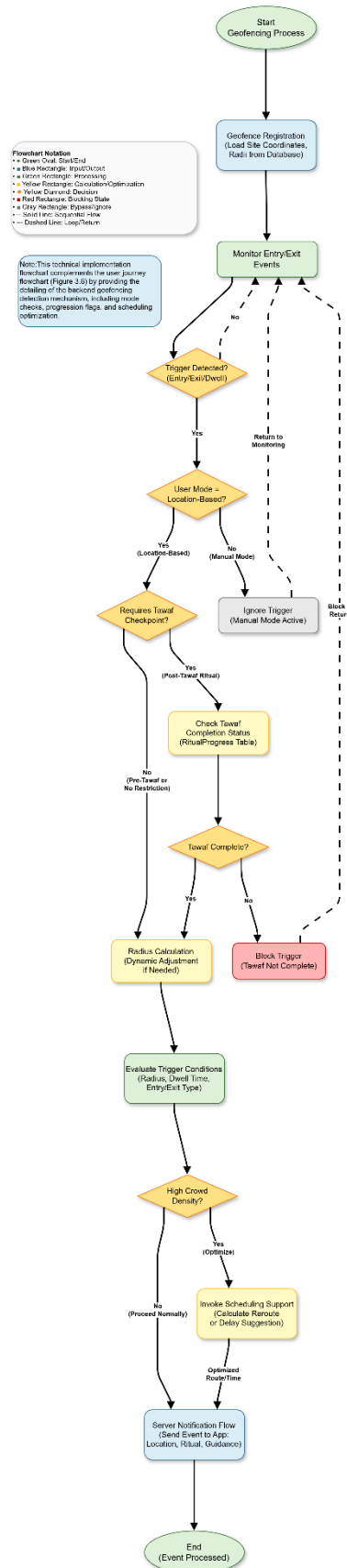


Figure 8: Geofencing logic flow

3.6 Database Design

The database structure is also designed to enable efficient, standard, and secure storage of all the application data, and it supports the third objective of the project, which is to streamline data management and performance, in connection, especially, with geofencing-based features and mode-specific functionality (GeeksforGeeks, 2024). The design is also scalable to real-time location queries with the use of geospatial elements and tracking of progression as well as data integrity is preserved in a mobile environment (Solutions Review, 2025).

3.6.1 Entity-Relationship Model

The Entities-Relationship Diagram (ERD) depicted in figure 8 represents the basic data structure and relationships among them in a Chen-style notation. There are core entities which are:

- a) Pilgrim, storing such personal information like name, email, phone, language preferences, health conditions, ModePreference (ENUM: 'Manual', 'Location-Based') and LocationPermissionStatus (ENUM: 'Granted', 'Denied', 'NotAsked').
- b) Schedule, ritual activity, planned and actual time, status, and priority; Location, sacred location, latitude, longitude, site type, site capacity, and geofence radius of default.
- c) RitualTemplates, which define the standard rituals by names, description, duration, flexibility, and order of sequence.
- d) RitualProgress, the composition of which is tracking ProgressID, PilgrimID, RitualTemplateID, CompletionStatus (ENUM: 'Pending', 'InProgress', 'Completed'), CompletedTimestamp, and ModeUsed to track ritual progress per mode.

The most important relationships are Pilgrim belongs to many Schedules (One-to-Many), Pilgrim has many RitualProgress entries (One-to-Many), and Schedule associates

with Location using a junction table to resolve the Many-to-Many relationship, to make it easy to query geofences. To address redundancy and update anomalies the model is normalized to reach at least the Third Normal Form (3NF) with the attributes in dependent relationships being broken down accordingly (GeeksforGeeks, 2024). Architecture is geospatially data-oriented, which guarantees a high level of support to a location-based trigger and user mode preferences within a mobile setting (Solutions Review, 2025).

Database Schema for Pilgrim Journey Management System

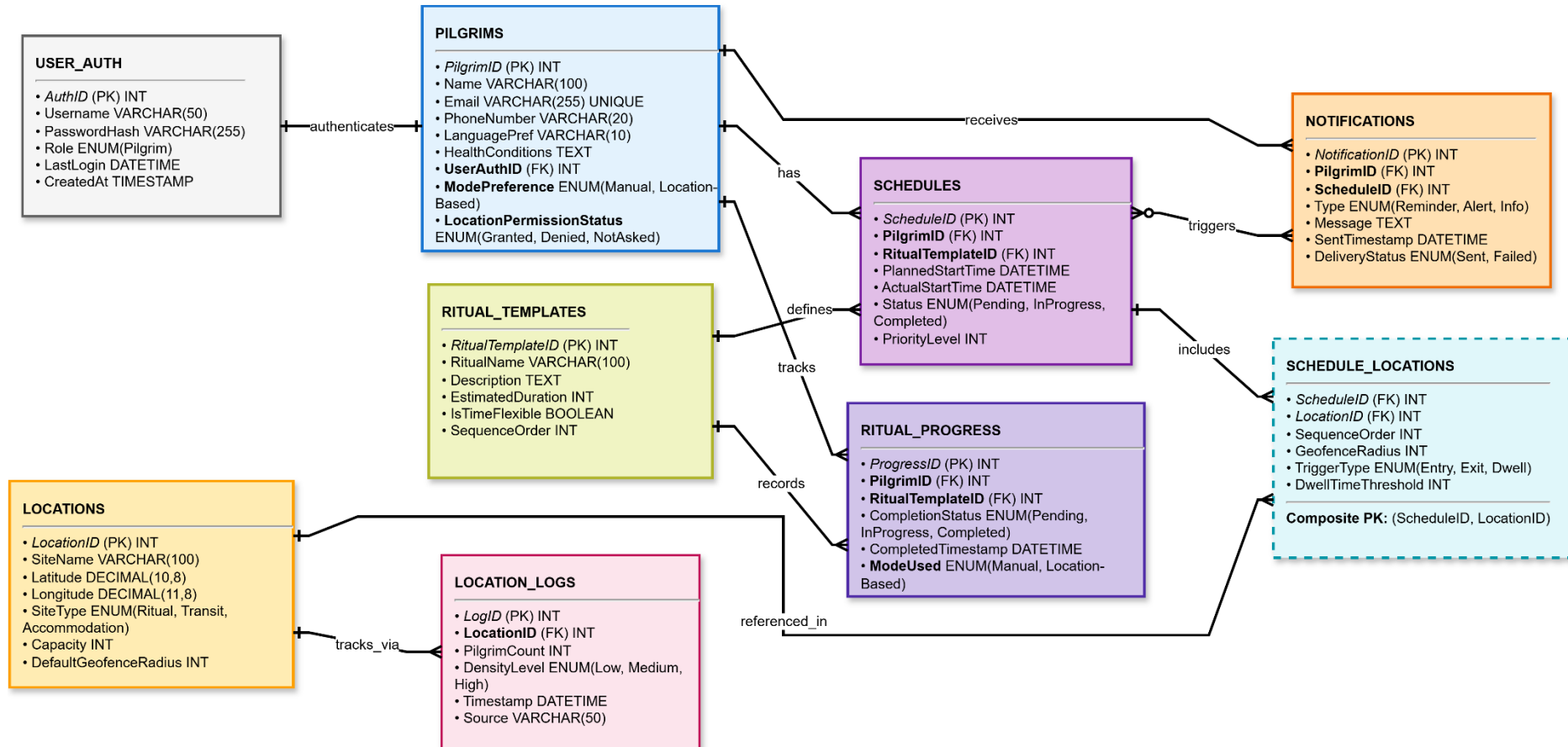


Figure 9: ERD diagram

3.6.2 Definitions of Schmas and Query Optimization Strategy.

The proposed database schema, based on the ERD, is outlined in Table 3 by the key attribute, constraint, primary key and foreign key. As an example, the PILGRIMS table will contain PilgrimID as a primary key, with additional modepreference and locationpermissionstatus attributes, and foreign key to USER_AUTH. The table RITUAL_PROGRESS has the ProgressID as the primary key, fields which are the foreign keys to PILGRIMS and RITUAL_TEMPLATES, and through which mode-specific tracking is provided.

The optimization of queries is also a part of meeting performance requirements of geofencing and dynamic modes, with strategic indexing. This strategic indexing includes composite indexes on (PilgrimID, ModePreference) to perform retrieval of rapid mode-filtered indexes and spatial indexes on the Latitude and Longitude of Location to perform efficient proximity query of geofence detection (GeeksforGeeks, 2024). Also, RITUAL_PROGRESS (PilgrimID, CompletionStatus) indexes are used to speed up progression checks (e.g., checking Tawaf completion before proceeding with rituals). In FYP2, EXPLAIN commands will be used to optimize and filter pathways of execution of complex joins involving Location, Schedule and RitualProgress tables, without slowing down the systems when many pilgrims are operating concurrently (Solutions Review, 2025). This is done to guarantee that data operations facilitate real-time decision-making and that the transactions are reliable in terms of location-based applications that follow the ACID principles of reliability in transactions (Vedantham, 2024).

Table 4: Proposed Database Schema.

Table Name	Key Attributes (Name: Data Type, Constraints)	Primary Key	Foreign Key(s)
USER_AUTH	AuthID: INT, NOT NULL Username: VARCHAR(50), UNIQUE PasswordHash: VARCHAR(255) Role: ENUM('Pilgrim') LastLogin: DATETIME CreatedAt: TIMESTAMP, DEFAULT CURRENT_TIMEST AMP	AuthID	-
PILGRIMS	PilgrimID: INT, NOT NULL Name: VARCHAR(100) Email: VARCHAR(255), UNIQUE PhoneNumber: VARCHAR(20) LanguagePref: VARCHAR(10) HealthConditions: TEXT UserAuthID: INT, NOT NULL ModePreference: ENUM('Manual', 'Location-Based') LocationPermissionSt atus: ENUM('Granted', 'Denied', 'NotAsked')	PilgrimID	UserAuthID → USER_AUTH
RITUAL_TEMPLATES	RitualTemplateID: INT, NOT NULL RitualName: VARCHAR(100) Description: TEXT EstimatedDuration: INT IsTimeFlexible: BOOLEAN	RitualTemplat eID	-

	SequenceOrder: INT		
SCHEDULES	ScheduleID: INT, NOT NULL PilgrimID: INT, NOT NULL RitualTemplateID: INT PlannedStartTime: DATETIME ActualStartTime: DATETIME Status: ENUM('Pending', 'InProgress', 'Completed') PriorityLevel: INT	ScheduleID	PilgrimID → PILGRIMS RitualTemplateID → RITUAL_TEMPLATES
LOCATIONS	LocationID: INT, NOT NULL SiteName: VARCHAR(100) Latitude: DECIMAL(10,8) Longitude: DECIMAL(11,8) SiteType: ENUM('Ritual', 'Transit', 'Accommodation') Capacity: INT DefaultGeofenceRadius: INT	LocationID	-
SCHEDULE_LOCATIONS	ScheduleID: INT, NOT NULL LocationID: INT, NOT NULL SequenceOrder: INT GeofenceRadius: INT TriggerType: ENUM('Entry', 'Exit', 'Dwell') DwellTimeThreshold: INT	(ScheduleID, LocationID)	ScheduleID → SCHEDULES LocationID → LOCATIONS
LOCATION_LOGS	LogID: INT, NOT NULL LocationID: INT, NOT NULL PilgrimCount: INT DensityLevel: ENUM('Low', 'Medium', 'High') Timestamp: DATETIME Source:	LogID	LocationID → LOCATIONS

	VARCHAR(50)		
NOTIFICATIONS	NotificationID: INT, NOT NULL PilgrimID: INT, NOT NULL ScheduleID: INT Type: ENUM('Reminder', 'Alert', 'Info') Message: TEXT SentTimestamp: DATETIME DeliveryStatus: ENUM('Sent', 'Failed')	NotificationID	PilgrimID → PILGRIMS ScheduleID → SCHEDULES
RITUAL_PROGRESS	ProgressID: INT, NOT NULL PilgrimID: INT, NOT NULL RitualTemplateID: INT, NOT NULL CompletionStatus: ENUM('Pending', 'InProgress', 'Completed') CompletedTimestamp: DATETIME ModeUsed: ENUM('Manual', 'Location-Based')	ProgressID	PilgrimID → PILGRIMS RitualTemplateID → RITUAL_TEMPLATES

3.7 Geofencing and Location Based Ritual Guidance.

Here, the technical details of geofencing as the central feature are described, and the issues of accuracy, battery life, and privacy are mentioned in the literature (Shevchenko and Reips, 2024). It theorizes the pilgrimage as a finite state machine, with geofence events triggering transitions between states (i.e., approaching Miqat to performing Tawaf), making sure that ritual state transitions are synchronized with user modes.

3.7.1 Technology Stack Native API Integration.

The application uses Flutter plugins to work with native APIs to access platform-specific optimizations needed to create reliable geofencing. In the case of Android, the GeofencingClient of Google Play Services is reached through the flutter_geofencing plug-in, whereby the definition of circular geofences along with the requested radius and PendingIntent callbacks of entering and leaving the geofence are established (Android Developers, 2024; Susanto et al., 2023). In iOS, the geolocator defined in the Core Location framework as a geolocator class, is used with the geolocator plug-in and offers delimiting opportunities on the data being tracked and the transitioned regions (Apple Developer, 2024). It has mode-specific behaviours in which in location-based mode, geofences are active and continuously tracked and enforced and in manual mode, they are disabled in order to save resources. The cross-platform quality of Flutter provides reliability in the implementation, and the state management widgets can easily help in changing modes with minimal latency in triggering rituals (Bhagat et al., 2025).

3.7.2 Configuration and Optimization

The balance between the precision and practicality of geofence is determined by a radius, where 50-100 meters is the general radius of sacred locations to allow GPS errors in densely populated cities such as the Haram (Shevchenko and Reips, 2024). Namely, ritual-specific modifications, such as a greater radius of Miqat points (150-200 meters) to ensure timely detection, 75-meter-wide Tawaf areas in the Mataf area, and long shapes (100m x 450m approximations) to designate Sa'i corridors, made use of 200-500 meters zones at Muzdalifah, Mina, and Arafat depending on the size of the

area (Volpis, 2025). Mode-specific behaviour is applied and in which location-based mode actively checks geofences and sequentially enforces them (e.g., Miqat before Tawaf), which causes automatic guidance and manual mode maintains the geofences in the inactive state and does not need to track the position.

Optimization uses fused location providers to make such smart choices of sources (GPS, Wi-Fi, cellular) to be used to obtain proper positioning with minimal power usage, and dynamically adjust the frequency of updates, with lower rates when the device is stationary (Manoj, 2025). Such issues as battery depletion and signal drift are addressed by adaptive polling and machine learning to predict the trajectory, saving up to 40% on GPS (Bhagat et al., 2025). This provides real-time response to triggers and FYP2 scalability testing.

Table 5: Ritual--Specific Geofence Configurations

Ritual Site	Geofence Radius	Justification
Miqat Points	150-200m	Early detection before entry
Tawaf (Mataf)	75m	Precise zone for sacred area
Sa'i Corridor	100m × 450m	Elongated rectangular path
Muzdalifah	200-500m	Large open area accommodation
Mina	200-500m	Extensive tent zone coverage
Arafat	200-500m	Vast plain monitoring

3.7.3 Implementation of Dual-Mode Architecture.

The mode selection and management are stored in a database (ModePreference in PILGRIMS table) and locally cached to allow offline access to allow real-time switching without restarting the app (Patil et al., 2025). When a user logs in, user preferences and permission status are recalled by the system, and only in location-based mode are geofencing services enabled in case permissions are allowed. Flutter state management is used to switch logic and deactivate/ activate Flutter monitors with a

smooth transition where fallback is used to manual mode when permissions are denied or revoked during the session.

In case of location-based mode, the progression is imposed using the RitualProgress table checks where Miqat/niyyah geofence triggers must be completed before Tawaf which is a gate that blocks access after Tawaf until a completion status is recorded. These are bypassed by manual mode, which lets one directly select a ritual. Permission management grants graceful degradation that means together with location-based will be able to fully track permissions; otherwise, will fall back to manual with user warnings (Vedantham, 2024).

3.7.4 Data Security Design and Privacy.

In a privacy-first design, all initial geofence detection occurs on the device, and only the necessary metadata of the events (e.g., entry timestamp) are sent to the server, as opposed to the raw coordinates. During onboarding, explicit consent is taken, and aggregated data to be used in analytics such as the crowd heatmaps are anonymized. All transmissions will use HTTPS/TLS encryption, which is GDPR / PDPA compliant. The user has the freedom to remove location history at any time or adjust the mode which guarantees control and respects ethics (Shevchenko and Reips, 2024).

3.8 Proposed Design Evaluation Strategy (FYP1/FYP2 Transition)

The artifacts created during FYP1 include the UML models, ERD, wireframes, the methodology plan are a design prototype, which is evaluated in a systematic manner to prove its validity and strength before the implementation of FYP2. This plan is based

on the principles of Design Science Research, which focuses on the iteration of the usefulness of the artifact and its alignment with the goals of problem-solving (Guracho et al., 2025).

Expert walkthroughs are conducted by domain experts, including expert pilgrimage guides and software architects, and are conducted systematically over diagrams and documents. The quality of the logical correctness of geofence-driven state transitions in location-based mode, the clarity of dual-mode architecture, the viability of ritual progression (e.g., Tawaf checkpoint enforcement), and adherence to Umrah processes will be assessed with the help of feedback. Moreover, usability of low-fidelity prototypes designed in Figma will be tested based on a minor group of prospective pilgrims. The preliminary perceptions will be collected with respect to the ease and friendliness of navigation, the ease of switching modes, the scenarios of permissions (e.g., graceful degradation in case of denying), and the usefulness of automatic displays of guidance, allowing user-driven improvements (Nikukaran et al., 2025). The transition in FYP2 will also involve the use of empirical validation with simulated and field tests, such as geofencing accuracy indicators (e.g., rates of precision of triggered modes), battery life analysis, and user satisfaction survey (e.g., modes preferences, effectiveness of ritual guidance).

3.9 Chapter Summary

This chapter has outlined an all-inclusive approach of the development of the Integrated Pilgrim Journey Management Application that focuses on geofencing-based ritual guidance as the core innovation. It was based on the Design Science Research paradigm and explained a three-level client-server architecture that is scalable, which is

justified by the choice of Flutter to build the front-end of the application and MySQL to manage the data stored on the back-end. The behaviour of the system was modelled with great precision using a set of UML diagrams: use cases, activities, sequences, and a user journey flowchart, with the focus made on the dual-mode architecture: unrestricted access via manual mode and enforced sequencing via location-based mode (Tawaf as an obligatory checkpoint). Designing the database was distilled to include mode preferences and ritual progression tracking without other features to isolate group coordination to focus on independent pilgrim functionality. The implementation of geofencing was described in technical terms (configuration, optimization, dual-mode logic, and privacy). Lastly, FYP1 artifact evaluation strategy was suggested, which will open the way to FYP2 validation and improvement.